



Facultad de Economía
Sede San Cristóbal

Unitrack Proyecto Final (Estructura de Datos)

Nombre de estudiante
Marcos Cifuentes

Carné
2527211

Guatemala, (junio) de 2026

MANUAL TÉCNICO

Proyecto UniTrack

Información General

Tecnologías Utilizadas:

- Angular
- TypeScript
- Node.js
- Express.js
- JavaScript
- Git
- GitHub

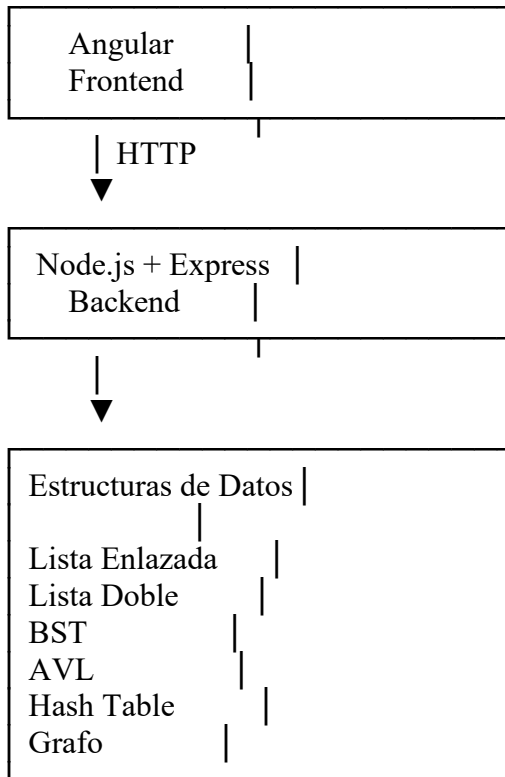
1. Introducción

UniTrack es un sistema de gestión académica desarrollado utilizando Angular para el frontend y Node.js con Express para el backend.

El objetivo principal del proyecto es aplicar e integrar diferentes estructuras de datos estudiadas durante el curso, permitiendo administrar estudiantes, historial académico, cursos, catedráticos y pensum académico mediante una interfaz web.

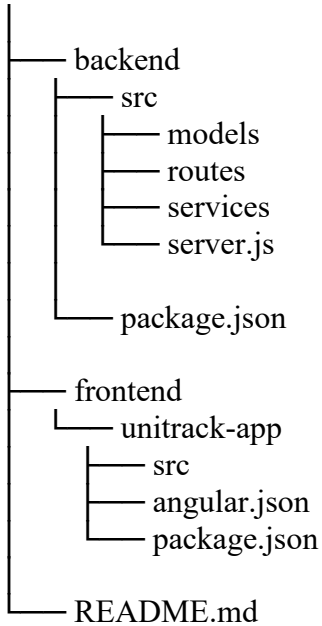
2. Arquitectura del Sistema

El sistema sigue una arquitectura Cliente-Servidor.



3. Estructura del Proyecto

UniTrack



3. Estructuras de Datos Implementadas

4.1 Lista Enlazada Simple

Módulo

Estudiantes

Descripción

Permite almacenar estudiantes de forma dinámica mediante nodos enlazados.

Operaciones

- Insertar estudiante
- Buscar estudiante
- Eliminar estudiante
- Listar estudiantes

Complejidad

Operación Complejidad

Insertar $O(1)$

Buscar $O(n)$

Eliminar $O(n)$

Recorrer $O(n)$

4.2 Lista Doblemente Enlazada

Módulo

Historial Académico

Descripción

Permite recorrer los registros tanto hacia adelante como hacia atrás.

Operaciones

- Insertar al inicio
- Insertar al final
- Mostrar normal
- Mostrar inverso

Complejidad

Operación	Complejidad
-----------	-------------

Insertar Inicio	$O(1)$
-----------------	--------

Insertar Final	$O(1)$
----------------	--------

Recorrer	$O(n)$
----------	--------

4.3 Árbol Binario de Búsqueda (BST)

Módulo

Cursos

Descripción

Los cursos se almacenan ordenados utilizando el código del curso como clave principal.

Operaciones

- Insertar
- Buscar
- Eliminar
- InOrden
- PreOrden
- PostOrden
- Obtener mínimo

- Obtener máximo
- Altura

Complejidad

Operación Promedio Peor Caso

Insertar $O(\log n)$ $O(n)$

Buscar $O(\log n)$ $O(n)$

Eliminar $O(\log n)$ $O(n)$

4.4 Árbol AVL

Descripción

Árbol binario balanceado que mantiene una altura controlada mediante rotaciones.

Operaciones

- Rotación simple izquierda
- Rotación simple derecha
- Rotación doble izquierda-derecha
- Rotación doble derecha-izquierda
- Inserción balanceada

Complejidad

Operación Complejidad

Insertar $O(\log n)$

Buscar $O(\log n)$

Eliminar $O(\log n)$

4.5 Tabla Hash

Módulo

Catedráticos

Descripción

Permite búsquedas rápidas mediante funciones hash.

Métodos Implementados

- Método División
- Método DJB2
- Encadenamiento
- Rehashing
- Linear Probing

Operaciones

- Insertar
- Buscar
- Eliminar
- Listar

Complejidad

Operación Complejidad Promedio

Insertar $O(1)$

Buscar $O(1)$

Eliminar $O(1)$

4.6 Grafo Dirigido

Módulo

Pensum Académico

Descripción

Representa cursos y prerrequisitos mediante una lista de adyacencia.

Operaciones

- Agregar curso
- Agregar prerrequisito
- BFS
- DFS
- Detección de ciclos
- Ordenamiento topológico
- Camino más corto
- Obtención de prerrequisitos

Complejidad

Operación	Complejidad
BFS	$O(V + E)$
DFS	$O(V + E)$
Topológico	$O(V + E)$
Detección de ciclos	$O(V + E)$

Donde:

- V = número de vértices
- E = número de aristas

4. API REST

Estudiantes

GET /api/estudiantes
POST /api/estudiantes
GET /api/estudiantes/:carnet
DELETE /api/estudiantes/:carnet

Historial Académico

GET /api/historial
GET /api/historial/inverso
POST /api/historial

Cursos

POST /api/cursos
GET /api/cursos/inorden
GET /api/cursos/preorden
GET /api/cursos/postorden
GET /api/cursos/minimo
GET /api/cursos/maximo
GET /api/cursos/altura
GET /api/cursos/:codigo
DELETE /api/cursos/:codigo

Catedráticos

GET /api/catedraticos
GET /api/catedraticos/factor-carga
POST /api/catedraticos
GET /api/catedraticos/:codigo
DELETE /api/catedraticos/:codigo

Pensum

POST /api/pensum/curso

POST /api/pensum/prerrequisito

GET /api/pensum

GET /api/pensum/bfs/:inicio

GET /api/pensum/dfs/:inicio

GET /api/pensum/topologico

GET /api/pensum/ciclos

5. Frontend

La interfaz fue desarrollada utilizando Angular.

Módulos implementados:

- Dashboard
- Estudiantes
- Historial
- Cursos
- Catedráticos
- Pensum

La comunicación con el backend se realiza mediante HttpClient y servicios REST.

6. Control de Versiones

El proyecto utiliza Git para el control de versiones y GitHub como repositorio remoto. Durante el desarrollo se realizaron múltiples commits para documentar el avance de cada estructura de datos y cada módulo implementado.

7. Conclusiones

El proyecto permitió aplicar de forma práctica múltiples estructuras de datos dentro de una aplicación web funcional.

La integración entre Angular y Node.js facilitó la construcción de un sistema modular capaz de representar problemas académicos mediante listas enlazadas, árboles, tablas hash y grafos.

Asimismo, se fortalecieron conocimientos relacionados con complejidad algorítmica, diseño de APIs REST y control de versiones mediante Git.